TP 9 : MINI ALGORITHME GÉNÉTIQUE QUI DEVINE UNE PHRASE CIBLE

Langage C++

Consigne : Ce TP est noté, le travail est à rendre sur Moodle et sera évalué à la fin de séance.

On souhaite écrire un programme itératif et évolutif qui **devine une phrase cible** en partant de chaînes de caractères aléatoires, qu'on améliore génération après génération (itération). À chaque génération, on garde les meilleures chaînes, on en croise deux pour fabriquer des « enfants », et on modifie parfois un caractère au hasard. Dans chaque génération (itération), une population de N individus est créée. Chaque individu est évalué à l'aide d'une fonction de perte (Fitness).

Boucle principale:

1. Générer une population de N individus chaqun ayant une chaîne aléatoire demême longueur que la phrase cible.
    - Chaque individu = une chaîne de caractères de même longueur que la cible.
    - Fitness = nombre de caractères corrects aux bonnes positions (ou une fonction équivalente très simple).
2. Répéter pour chaque génération :
    - Calculer un **score (fitness)** pour chaque individu (nombre de caractères corrects à la bonne position).
    - Garder toujours les meilleurss individu, ceux qui ont la fitness la plus élevée (élitisme).
    - Pour fabriquer une nouvelle population :
        - choisir N parents au hasard parmi les meilleurs,
        - créer un enfant en prenant une moitié du premier parent et une moitié du deuxième (croisement des deux chaines),
        - pour chaque caractère de l'enfant, avec une petite probabilité p, le remplacer par un caractère aléatoire (mutation),
        - ajouter l'enfant dans la nouvelle population.
    - Afficher / enregistrer : numéro de génération, meilleur score, score moyen.
    - Si une chaîne a le score maximal (tous les caractères corrects), on s'arrête, si non on continue sur maximum 1000 générations.

Le programme devra logger dans un fichier CSV à chaque génération :

- Les métriques globales : génération, best fitness, fitness moyenne, etc.
- (Optionnel / bonus) L'ensemble de la population avec fitness dans un autre fichier.

I. Fichier de Configuration

Un fichier de configuration texte est lu et interprété par le programme afin d'initialiser la première génération.

```
TARGET=HELLO WORLD
POP_SIZE=30
MAX_GEN=1000
MUTATION_RATE=0.05
LOG_POP_FILE=population.csv
LOG_STATS_FILE=stats.csv
```

II. Architecture du Programme

Implémentez le programme complet en vous basant sur cette architecture des classes.

1. Configuration

Classe « Config » :

```
class Config {
    std::string target;
    int popSize;
    int maxGenerations;
    double mutationRate;
    std::string logPopFile;
    std::string logStatsFile;

public:
    const std::string& getTarget() const;
    int getPopSize() const;
    int getMaxGenerations() const;
    double getMutationRate() const;
    const std::string& getLogPopFile() const;
    const std::string& getLogStatsFile() const;

    static Config fromFile(const std::string& filename);
};
```

Exceptions personnalisées :

```
class FichierIntrouvable : public std::exception { ... };
class FormatInvalide : public std::exception { ... };
```

2. Individu, Population, Fitness

Classe « Individu » :

```
class Individu {
    std::string genome;   // même longueur que target
    int fitness = 0;

public:
    explicit Individu(const std::string& genome);

    const std::string& getGenome() const;
    int getFitness() const;
    void setFitness(int f);

    static Individu randomIndividu(size_t length);

    // facultatif : opérateur < pour trier par fitness
};
```

- Génère une chaîne aléatoire de longueur *length*

Classe « Population » :

```
class Population {
    std::vector<Individu> individus;

public:
    Population() = default;
    explicit Population(const std::vector<Individu>& indivs);

    std::vector<Individu>& getIndividus();
    const std::vector<Individu>& getIndividus() const;

    const Individu& getBest() const;
    double getMoyenneFitness() const;
};
```

- getBest() : retourne l'individu avec fitness max.
- getMoyenneFitness() : moyenne des fitness.

3. Algorithme génétique et logging

Classe « Logger » :

```
class Logger {
    std::ofstream statsFile;
```

```
    std::ofstream popFile;

public:
    Logger(const std::string& statsFilename,
           const std::string& popFilename);

    ~Logger();

    void logStats(int generation, int bestFitness,
                  double avgFitness);

    void logPopulation(int generation,
                       const Population& pop);
};
```

Classe « GeneticAlgorithm » :

```
class GeneticAlgorithm {
    Config config;
    Logger logger;

    Population population;
    std::string target;

public:
    explicit GeneticAlgorithm(const Config& cfg);

    void initialiserPopulation();
    void evaluerPopulation();
    Individu selectionParent();
    Individu crossover(const Individu& p1, const Individu& p2);
    void mutation(Individu& indiv);

    void run();
};
```

- initialiserPopulation() : crée config.getPopSize() individus aléatoires
- evaluerPopulation() : pour chaque individu calculuer sa fitness
- selectionParent() : tournoi de taille 2 ou 3 (sélectionner 2 ou 3 individus au hasard et garder le meilleur), N tournois afin de sélectionner N parents
- crossover(p1, p2) :
    - croisement 1 point :
        - choisir un point de coupe k,
        - enfant = p1[0..k] + p2[k..end].
- mutation(indiv) :
    - pour chaque position du génome : avec probabilité mutationRate, remplacer par un caractère aléatoire.
- run() : boucle pour g de 0 à maxGenerations - 1 :
    - evaluerPopulation()

- Log des stats + population
- Si un individu a fitness == longueur de la cible alors on peut s'arrêter (solution trouvée).
- Créer une nouvelle population de taille identique à la première en répétant les opérations suivantes:
  - selectionParent deux fois,
  - crossover,
  - mutation,
  - ajouter l'enfant à la nouvelle population.
- remplacer l'ancienne population.